

PYTHON AUTOMATION SCRIPT

Internship Task 1 :

1. Title

Python Automation Script for File Organization and Renaming

2. Objective

The objective of this project is to automate file management operations using Python.

The script organizes files into different folders based on file type, renames files automatically, generates logs for operations performed, and handles errors using exception handling.

3. Problem Statement

Managing files manually can be time-consuming and inefficient.

This automation script helps organize files automatically by sorting them into folders such as Text, Images, and PDF.

The script also keeps track of all operations in a log file.

4. Features Implemented

- File sorting automation
- Automatic file renaming
- File movement operations
- User input support
- Logging system
- Exception handling
- OS module usage

5. Project Structure

Task1_Automation/

— automation.py

— logs.txt

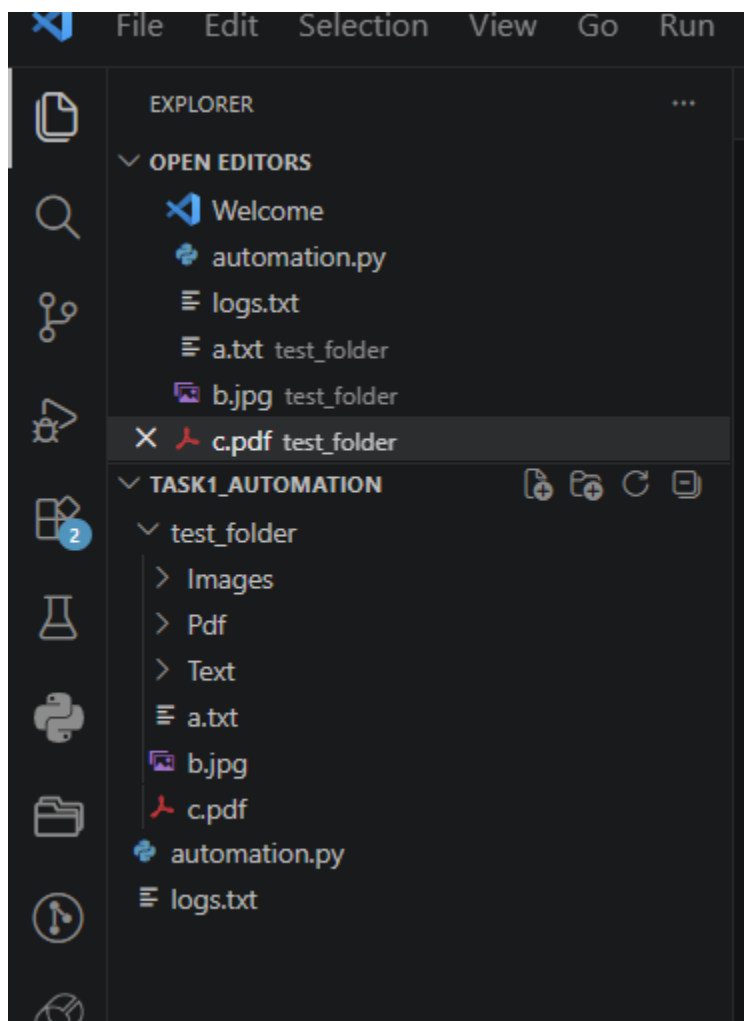
└─ test_folder/

— Text/

— Images/

— PDF/

6.Folder Before



7.code

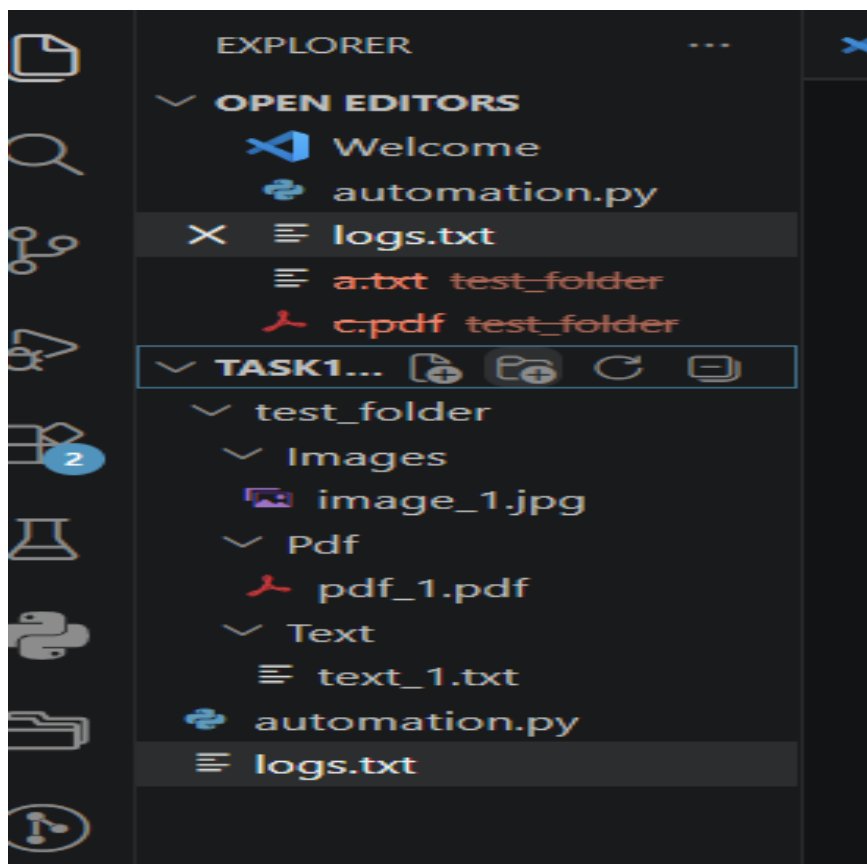
```
import os
import shutil
folder_path = input("Enter folder name: ")
def write_log(message):
    with open("logs.txt", "a") as log:
        log.write(message + "\n")
try:
    files = os.listdir(folder_path)
    text_count = 1
    image_count = 1
    pdf_count = 1
    for file in files:
        source = os.path.join(folder_path, file)
        if os.path.isfile(source):
            if file.endswith(".txt"):
                new_name = f"text_{text_count}.txt"
                destination = os.path.join(
                    folder_path,
                    "Text",
                    new_name
                )
                shutil.move(source, destination)
                write_log(f"{file} moved to Text as {new_name}")
                text_count += 1
            elif file.endswith(".jpg"):
                new_name = f"image_{image_count}.jpg"
                destination = os.path.join(
                    folder_path,
                    "Images",
                    new_name
                )
                shutil.move(source, destination)
                write_log(f"{file} moved to Images as {new_name}")
                image_count += 1
            elif file.endswith(".pdf"):
                new_name = f"pdf_{pdf_count}.pdf"
                destination = os.path.join(
                    folder_path,
                    "PDF",
                    new_name
                )
                shutil.move(source, destination)
                write_log(f"{file} moved to PDF as {new_name}")
                pdf_count += 1
    print("Files organized successfully!")
except Exception as e:
    print("Error:", e)
    write_log(f"ERROR: {e}")
```

8.Terminal Output

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  SPELL CHECKER

• PS C:\Users\navya\task1_Automation> python automation.py
  Enter folder name: test_folder
  Files organized successfully!
❖ PS C:\Users\navya\task1_Automation> |
```

9.Folder after



10.Log file

```
logs.txt
1  a.txt moved to Text as text_1.txt
2  b.jpg moved to Images as image_1.jpg
3  c.pdf moved to PDF as pdf_1.pdf
4
```

11. Explanation of the Code

Importing Modules

- os module is used for file and folder operations.
- shutil module is used for moving files.

User Input

The program asks the user to enter the folder name.

Logging

A log file named logs.txt is created to store all operations performed by the script.

File Sorting

The script checks file extensions:

- .txt → Text folder
- .jpg → Images folder
- .pdf → PDF folder

File Renaming

Files are renamed automatically using counters:

- text_1.txt
- image_1.jpg
- pdf_1.pdf

Exception Handling

try-except is used to prevent program crashes and handle errors safely.

12.Conclusion

This project successfully demonstrates Python automation using file handling, renaming, sorting, logging, and exception handling. The automation script improves file management efficiency and reduces manual effort.'

13. Future Enhancements

- Support more file types
- GUI interface using Tkinter
- Automatic scheduled execution
- Duplicate file detection
- Cloud storage integration